INSTITUTO TECNOLÓGICO DE ESTUDIOS

SUPERIORES DE OCCIDENTE

Práctica #1: Creación de Corpus

por

César Villarreal Hernández, 707560

Minería de Textos

Dr. Víctor Hugo Zaldivar Carrillo

Miércoles 18 de febrero de 2026

I. Objetivo técnico

El objetivo técnico de esta práctica consiste en construir un corpus de artículos periodísticos utilizando la API de The Guardian. “se llama *corpus* a un **conjunto formado por miles de textos** (novelas, obras de teatro, guiones de cine, noticias de prensa, ensayos, transcripciones de noticiarios radiofónicos o televisivos, transcripciones de conversaciones, discursos, etc.) y cientos o miles de millones de formas.” (Real Academia Española, s. f.).

II. Características generales

Para la creación del corpus se utilizó se utilizó la API oficial de The Guardian para realizar solicitudes de HTTP y obtener los textos en formato JSON. El script se encuentra conformado principalmente por tres archivos de Python: corpus.py, guardian.py, y workspace.py. El script corpus consiste en el programa principal y en este se manda a llamar la API de Guardian por medio de un clase que construí a partir de la API de The Guardian.

-

```
src > corpus.py > ...
# Importamos librerías
import json
import requests
import os
from os import makedirs
from os.path import join, exists
from datetime import date, timedelta
import ast
import workspace
import sys
from guardian import Guardian

def __main__():
    os.chdir("..")
    # retrieve the workspace path
    workspace.set_workspace_path(os.getcwd())

    start_date = date(2026,1,1)
    end_date = date(2026,2,1)

    # create object for the Guardian API
    g = Guardian(start_date, end_date)
    # retrieve articles from the start date to the end date
    g.fetch_articles()

__main__()
```

El objetivo de principal de nuestros scripts es el de descargar diversos textos y almacenarlos en diferentes carpetas de acuerdo con la categoría de los diferentes artículos.

Para facilitar el uso del script creado en futuros proyectos, decidí implementarlo como una clase con diferentes funciones. De manera que al constructor de la clase se le pasa una fecha de inicio y una fecha de fin para filtrar por fecha los artículos que se desean descargar, así como el límite de estos.

```
class Guardian():
    # base url for the search
    API_ENDPOINT = "http://content.guardianapis.com/search"

    # parameter dictionary for the search
    api_params = {}

    key_api = ""

    start_date = date
    end_date = date

    def __init__(self, start_date:date, end_date:date):
        # retrieve the guardian key string
        self.key_api = self.read_guardian_api_key()
        self.start_date = start_date
        self.end_date = end_date
        # initialize dictionary for parameter search
        self.api_params = {
            'from-date':"",
            'to-date':"",
            'order-by':"newest",
            'show-fields': "bodyText",
            'page-size': 200,
            'api-key': self.key_api
        }
```

Se creo una función para realizar la descarga de los artículos llamada “fetch_articles”. Esta función tiene toma las fechas de inicio y fin de la clase Guardian, y realiza la descarga de los artículos en esas fechas. Antes de realizar la descarga detecta si el archivo

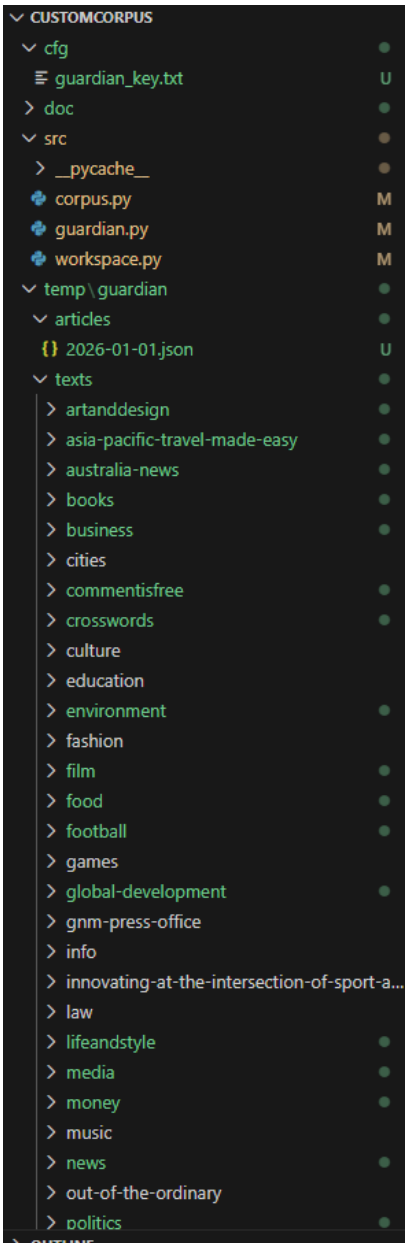
JSON con la fecha de inicio ya existe, y procede a leer el archivo; de otra manera realiza la descarga de los documentos y crea este archivo JSON.

```python
def fetch_articles(self, debug = 1):
    resultados = []
    # string date conversion
    start_date_str = self.start_date.strftime("%Y-%m-%d")
    end_date_str = self.end_date.strftime("%Y-%m-%d")
    # article file name
    filename = os.path.join(workspace.get_articles_path(), start_date_str + '.json')
    total_pages = 200

    if(not os.path.exists(filename)):
        if(debug): print("\nDownloading pages for %s\n" % filename)
        print(start_date_str)
        self.api_params['from-date'] = start_date_str
        self.api_params['to-date'] = end_date_str

        current_page = 1
        # estamos tomando páginas de 200 artículos

        while current_page <= total_pages:
            print("...page", current_page)
            self.api_params['page'] = current_page
            resp = requests.get(self.API_ENDPOINT, self.api_params)
            data = resp.json()
            resultados.extend(data['response']['results'])
            # if there is more than one page
            current_page += 1
            total_pages = data['response']['pages']
            print(current_page)

        # store the result in a json file
        with open(filename, 'w') as f:
            print("\nWriting to", filename)
            # hacemos un dump para que se imprima bonito
            f.write(json.dumps(resultados, indent=2))

        self.process_json_pages(filename, start_date_str)
    else:
        if(debug): print("\nFile %s already exists\n" %filename)

        with open(filename, 'r') as f:
            lines = f.readlines()

            if("[]" in lines or (len(lines) == 0)):
                if(debug): print("\n> File %s is empty..., re-fetching files\n" %filename)
                f.close()
```

Posteriormente, si el archivo existe se ejecuta la función “process_json_pages”. Esta función procesa el archivo JSON y guarda la información en un archivo CSV.

```python
def fetch_articles(self, debug = 1):
        self.process_json_pages(filename, start_date_str)
    else:
        if(debug): print("\nFile %s already exists\n" %filename)

        with open(filename, 'r') as f:
            lines = f.readlines()

            if("[]" in lines or (len(lines) == 0)):
                if(debug): print("\n> File %s is empty..., re-fetching files\n" %filename)
                f.close()
                os.remove(filename)
                self.fetch_articles(self.start_date, self.end_date, total_pages, debug)
            else:
                print(filename)
                self.process_json_pages(filename, start_date_str)
                self.process_articles_to_text_csv()


def process_json_pages(self, file_name, start_date_str, debug = 1):
    titulos = []
    articulos = []
    csv_struct = []
    section_id = []

    with open(file_name, "r") as f:
        contenido = json.load(f)
        i = 0
        for page in contenido:
            titulo = page["webTitle"]
            titulos.append(page["webTitle"])

            articulo = page["fields"]["bodyText"]
            articulos.append(articulo)

            section = page["sectionId"]
            section_id.append(section)

            csv_struct.append([i, titulo, articulo, section])
            columnas = ["id", "Web Title", "Body Text", "SecId"]
            df = pd.DataFrame(csv_struct, columns=columnas)
            csv_file_name = "articulos-" + start_date_str
            page_csv_path = os.path.join(workspace.get_texts_path(), (csv_file_name+ ".csv"))
            df.to_csv(page_csv_path, index=False, encoding="utf-8-sig")

            if(debug): print("%i. %s" % (i, titulo))
            i = i + 1
```

Después de esto, se realiza la ejecución de la función “process_articles_to_text_csv”, la cual tiene como propósito leer el archivo CSV que fue guardado en el directorio temp/guardian/texts, y guarda los artículos en diferentes carpetas que van de acuerdo a la sección del artículo.



III. Alcances y limitaciones

Se logró generar el corpus correctamente, de manera de que los artículos se encuentran separados en diferentes carpetas y de acuerdo con su sección. Este script me permitirá obtener información periodística actualizada con la cual podré realizar el procesamiento de textos en futuras prácticas.

IV. Conclusiones

En conclusión se puede afirmar que se alcanzaron los objetivos principales de esta práctica, de manera que se pudo crear un corpus a partir de la API de Guardian con la cual se podrá realizar el procesamiento de texto a futuro.

V. Bibliografía

- Real Academia Española. (s. f.). *CORPES XXI*. https://www.rae.es/banco-de-datos/corpes-xxi